QualiTest — L2 Technical Deep-Dive (Part 1)

CI/CD Pipelines + GitLab + ArgoCD + Artifactory

> **Role:** Senior DevOps/SRE Engineer | **Round:** L2 Technical (1 Hour) **Must-Haves:** CI/CD (GitLab, ArgoCD, Artifactory), Linux & Scripting, Terraform

SECTION 1: CI/CD PIPELINE ARCHITECTURE (Core Focus — 4/5 Rating Required)

Q1. Walk us through how you design a production CI/CD pipeline end-to-end.

Answer:

My standard production pipeline architecture:

```
Developer → GitLab (merge request) → CI Pipeline → Artifactory → CD Pipeline
→ Production
```

Detailed stages:

```
stages:
  - validate        # Lint, static analysis, security scan
  - build           # Compile, Docker build
  - test            # Unit tests, integration tests, coverage
  - security        # SAST, DAST, dependency scan, container scan
  - publish         # Push artifacts to Artifactory / container images to
registry
  - deploy-dev      # Auto-deploy to dev
  - deploy-staging  # Auto-deploy to staging + run E2E tests
  - deploy-prod     # Manual approval → ArgoCD sync to production
```

Key design principles I follow:

1. **Fail fast:** Lint and static analysis run first (30 seconds). No point running 15-minute test suites if code doesn't compile
2. **Immutable artifacts:** Build once, promote everywhere. The exact same Docker image deployed to dev is promoted to staging and prod via Artifactory
3. **Security as a gate, not an afterthought:** SAST/DAST/container scanning runs before publish. Critical vulnerabilities block the pipeline

4. **Environment parity:** Dev, staging, prod are identical infrastructure (Terraform modules with different tfvars)
5. **Rollback capability:** Every deployment is versioned. ArgoCD can rollback to any previous Git commit in seconds

---

Q2. Explain your hands-on experience with GitLab CI/CD. How do you structure `.gitlab-ci.yml` for a complex project?

Answer:

My GitLab CI structure for a microservices project:

```
# .gitlab-ci.yml
include:
  - local: '/ci/templates/docker-build.yml'    # Reusable templates
  - local: '/ci/templates/terraform.yml'
  - local: '/ci/templates/security-scan.yml'

variables:
  DOCKER_REGISTRY: "company.jfrog.io"
  IMAGE_TAG: "${CI_COMMIT_SHORT_SHA}"
  TF_STATE_NAME: "${CI_PROJECT_NAME}"

stages:
  - validate
  - build
  - test
  - security
  - publish
  - deploy

# ---- VALIDATE ----
lint:
  stage: validate
  image: python:3.11-slim
  script:
    - pip install flake8 black
    - black --check src/
    - flake8 src/ --max-line-length=120
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

terraform-validate:
  stage: validate
  image: hashicorp/terraform:1.7
  script:
```

```
    - cd infra/ && terraform init -backend=false
    - terraform validate
    - terraform fmt -check -recursive

# ---- BUILD ----
docker-build:
  stage: build
  image: docker:24-dind
  services:
    - docker:24-dind
  script:
    - docker build -t ${DOCKER_REGISTRY}/app:${IMAGE_TAG} .
    - docker save ${DOCKER_REGISTRY}/app:${IMAGE_TAG} > app.tar
  artifacts:
    paths: [app.tar]
    expire_in: 1 hour

# ---- TEST ----
unit-tests:
  stage: test
  image: python:3.11
  script:
    - pip install -r requirements.txt
    - pytest tests/ -v --cov=src --cov-report=xml
  coverage: '/TOTAL.*\s+(\d+%)/'
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml

# ---- SECURITY ----
container-scan:
  stage: security
  image: aquasec/trivy:latest
  script:
    - trivy image --severity HIGH,CRITICAL --exit-code 1
${DOCKER_REGISTRY}/app:${IMAGE_TAG}

sast:
  stage: security
  include:
    - template: Security/SAST.gitlab-ci.yml

# ---- PUBLISH ----
push-to-artifactory:
  stage: publish
  script:
    - docker load < app.tar
    - docker login ${DOCKER_REGISTRY} -u ${JFROG_USER} -p ${JFROG_TOKEN}
    - docker push ${DOCKER_REGISTRY}/app:${IMAGE_TAG}
    # Tag as latest for the branch
    - docker tag ${DOCKER_REGISTRY}/app:${IMAGE_TAG}
${DOCKER_REGISTRY}/app:latest
    - docker push ${DOCKER_REGISTRY}/app:latest
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

```yaml
# ---- DEPLOY ----
deploy-dev:
  stage: deploy
  environment:
    name: development
  script:
    - |
      # Update ArgoCD application image tag
      cd k8s-manifests/overlays/dev/
      kustomize edit set image app=${DOCKER_REGISTRY}/app:${IMAGE_TAG}
      git add . && git commit -m "deploy: dev ${IMAGE_TAG}"
      git push origin main
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

deploy-prod:
  stage: deploy
  environment:
    name: production
  script:
    - cd k8s-manifests/overlays/prod/
    - kustomize edit set image app=${DOCKER_REGISTRY}/app:${IMAGE_TAG}
    - git add . && git commit -m "deploy: prod ${IMAGE_TAG}"
    - git push origin main
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual  # Manual approval gate for production
```

Advanced GitLab features I use:

Feature	Purpose
Parent-child pipelines	Trigger separate pipelines per microservice in a monorepo
DAG (needs keyword)	Parallel execution — tests don't wait for unrelated builds
Rules + workflow	Fine-grained control — only run deploy on `main`, run lint on MRs
GitLab Environments	Track what's deployed where, with rollback buttons
Protected variables	Prod credentials only available in protected branch pipelines
Cache + artifacts	Cache `node_modules`/`pip` between runs; pass build artifacts between stages

Feature	Purpose
Review Apps	Auto-deploy MR branches for QA review, auto-destroy on merge

Q3. Explain ArgoCD in depth. How does GitOps work and why is it better than push-based CD?

Answer:

ArgoCD is a declarative, GitOps-based continuous delivery tool for Kubernetes.

Core concept: Git is the single source of truth for the desired state of your infrastructure. ArgoCD continuously monitors Git and reconciles the live cluster state to match.

Push vs Pull CD:

Aspect	Push-based (GitLab CD, Jenkins)	Pull-based (ArgoCD GitOps)
Who deploys	CI server pushes to cluster	ArgoCD in-cluster pulls from Git
Credentials	CI needs cluster credentials (security risk)	ArgoCD already has cluster access
Drift detection	None — manual `kubectl` changes persist	ArgoCD detects and auto-corrects drift
Audit trail	CI logs	Git history = complete audit trail
Rollback	Re-run old pipeline	`git revert` → ArgoCD auto-syncs

ArgoCD architecture I implement:

```
Git Repository (k8s-manifests/)
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
```

```
├── overlays/
│   ├── dev/
│   │   └── kustomization.yaml (replica: 1, dev image tag)
│   ├── staging/
│   │   └── kustomization.yaml (replica: 2, staging tag)
│   └── prod/
│       └── kustomization.yaml (replica: 5, prod tag, resources)
```

ArgoCD Application manifest:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: payment-service
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://gitlab.company.com/k8s-manifests.git
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: payments
  syncPolicy:
    automated:
      prune: true          # Delete resources removed from Git
      selfHeal: true       # Revert manual kubectl changes
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 3
      backoff:
        duration: 5s
        factor: 2
```

Key ArgoCD features I leverage:

- **Sync Waves:** Deploy CRDs before applications, secrets before deployments
- **Health checks:** Custom health assessments for CRDs
- **App of Apps pattern:** One ArgoCD Application manages all other Applications — entire platform defined in Git
- **ApplicationSets:** Template-based generation — one definition creates apps across 10 environments
- **Notifications:** Slack/Teams alerts on sync success/failure
- **RBAC:** Team A can sync their apps only, not Team B's

Q4. What is Artifactory and how do you integrate it in CI/CD?

Answer:

JFrog Artifactory is a universal artifact repository manager. It stores every type of build artifact — Docker images, Helm charts, Python packages, Maven JARs, npm packages, Terraform modules, and generic binaries.

Why Artifactory over simple registries:

Feature	Docker Hub / ECR	Artifactory
Multi-format	Container images only	Docker + Helm + npm + PyPI + Maven + generic
Promotion	Manual re-tag	Built-in promotion (dev → staging → prod)
Security scanning	Basic	JFrog Xray — deep CVE and license scanning
Metadata	Tags only	Custom properties, build info, dependency graph
Replication	Region-based	Multi-site, multi-cloud replication

My Artifactory repository structure:

```
Artifactory
├── docker-local-dev/        # Dev images (auto-cleanup after 30 days)
├── docker-local-staging/    # Promoted images for staging
├── docker-local-prod/       # Promoted images for production (retained 1
year)
├── docker-remote/           # Proxy for Docker Hub (caching, security)
├── helm-local/              # Internal Helm charts
├── pypi-local/              # Internal Python packages
├── pypi-remote/             # Proxy for PyPI
├── terraform-local/         # Internal Terraform modules
└── generic-local/           # Build artifacts, config bundles
```

Promotion workflow (immutable artifacts):

```
# In GitLab CI — build and push to dev repo
docker push company.jfrog.io/docker-local-dev/app:${SHA}

# After staging tests pass — promote (copy, don't rebuild)
jfrog rt cp \
  docker-local-dev/app/${SHA}/ \
  docker-local-staging/app/${SHA}/

# After prod approval — promote to prod repo
jfrog rt cp \
  docker-local-staging/app/${SHA}/ \
  docker-local-prod/app/${SHA}/
```

Key principle: The image deployed to production is the **exact same binary** built during CI. No rebuilding. Artifactory promotion ensures bit-for-bit identity.

Integration with ArgoCD:

- ArgoCD pulls Helm charts from Artifactory's Helm repo
- Image tags in Kustomize overlays reference Artifactory Docker repos
- Xray webhook blocks deployment if critical CVE found post-publish

---

Q5. How do you handle secrets in CI/CD pipelines?

Answer:

Rule #1: Never store secrets in Git. Ever.

Secrets management strategy by layer:

CI Pipeline (GitLab):

- **GitLab CI/CD Variables:** Masked + protected variables for API keys, tokens
- **Protected variables:** Only available on protected branches (main/release) — a developer's feature branch pipeline cannot access prod credentials
- **File-type variables:** For certificates and kubeconfig files
- **Variable scoping:** Different values per environment (dev/staging/prod)

Infrastructure (Terraform):

- **AWS Secrets Manager** for application secrets (DB passwords, API keys)
- **Terraform state:** Encrypted S3 backend — state contains sensitive outputs
- `sensitive = true` flag on Terraform variables/outputs — prevents console display
- **Never** `terraform output` **secrets in CI logs**

Kubernetes (ArgoCD):

- **External Secrets Operator (ESO):** Syncs AWS Secrets Manager → K8s Secrets automatically

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials
  data:
    - secretKey: password
      remoteRef:
        key: prod/database/password
```

- **Sealed Secrets** as an alternative — encrypt secrets in Git, only the cluster can decrypt
- **Never** use plain K8s Secrets in Git repos (base64 ≠ encryption)

SECTION 2: LINUX & SCRIPTING (4/5 Rating Required)

Q6. A production server is running slow. Walk through your Linux troubleshooting process.

Answer:

My systematic approach — "USE Method" (Utilization, Saturation, Errors):

```
# Step 1: Overview — what's happening right now?
uptime                    # Load average (1/5/15 min)
top -bn1 | head -20       # CPU, memory, top processes
free -h                   # Memory usage and swap
df -h                     # Disk space
iostat -x 1 3             # Disk I/O (await, %util)

# Step 2: CPU deep-dive
mpstat -P ALL 1 3         # Per-CPU utilization
pidstat -u 1 5            # Per-process CPU usage
# High %usr → application code; High %sys → kernel/syscalls; High %iowait →
disk bottleneck

# Step 3: Memory deep-dive
vmstat 1 5                # si/so (swap in/out) — if non-zero, you're
swapping
cat /proc/meminfo | grep -i "MemAvailable\|Buffers\|Cached\|SwapUsed"
slabtop                   # Kernel memory allocation

# Step 4: Disk I/O
iotop -ao                 # Which processes are doing I/O
iostat -xz 1 5            # %util > 80% = disk saturated; await > 10ms =
slow disk
lsof +D /var/log          # Who's writing to /var/log (log explosion?)

# Step 5: Network
ss -tulnp                 # Open ports and connections
ss -s                     # Connection state summary (TIME_WAIT flooding?)
nethogs                   # Per-process bandwidth
sar -n DEV 1 5            # Network throughput

# Step 6: Process-level investigation
strace -p <PID> -c        # System call summary (what's the process doing?)
perf top                  # CPU profiling — which functions are hot
/proc/<PID>/fd | wc -l    # File descriptor count (leak?)
```

Common root causes and fixes:

Symptom	Likely Cause	Fix
High load, low CPU	I/O wait — disk or network blocked	Check `iostat`, move to SSD/EBS gp3
Memory growing over time	Memory leak	Identify process, restart, fix code
Swap usage increasing	OOM pressure	Add memory or reduce workload

Symptom	Likely Cause	Fix
High %sys CPU	Too many context switches or syscalls	Check `vmstat` cs column, tune app
Connection timeouts	File descriptor exhaustion	Check `ulimit -n`, increase if needed
Disk full	Log files or temp files	Find large files: `find / -xdev -size +100M`

Q7. Write a bash script to monitor disk usage and alert when it exceeds 80%.

Answer:

```
#!/bin/bash
# disk_monitor.sh — Production disk usage monitor with alerting
# Usage: Run via cron every 5 minutes: */5 * * * *
/opt/scripts/disk_monitor.sh

set -euo pipefail

THRESHOLD=80
ALERT_WEBHOOK="https://hooks.slack.com/services/XXX/YYY/ZZZ"
HOSTNAME=$(hostname -f)
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
LOG_FILE="/var/log/disk_monitor.log"

send_alert() {
    local mount="$1" usage="$2" avail="$3"
    local message=":warning: *Disk Alert on ${HOSTNAME}*\n• Mount:
\`${mount}\`\n• Usage: *${usage}%*\n• Available: ${avail}\n• Time:
${TIMESTAMP}"

    curl -s -X POST "${ALERT_WEBHOOK}" \
        -H 'Content-Type: application/json' \
        -d "{\"text\": \"${message}\"}" >> "${LOG_FILE}" 2>&1
}

# Parse df output, skip header and tmpfs/devtmpfs
df -h --output=pcent,avail,target -x tmpfs -x devtmpfs | tail -n +2 | while
read -r usage avail mount; do
    # Strip % sign
    usage_num=${usage%%%}

    if [[ ${usage_num} -ge ${THRESHOLD} ]]; then
        echo "${TIMESTAMP} ALERT: ${mount} at ${usage_num}% (avail:
${avail})" >> "${LOG_FILE}"
```

```
            send_alert "${mount}" "${usage_num}" "${avail}"

            # Auto-remediation: clean old logs if /var/log is the culprit
            if [[ "${mount}" == "/" || "${mount}" == "/var" ]]; then
                find /var/log -name "*.gz" -mtime +7 -delete 2>/dev/null
                journalctl --vacuum-time=3d 2>/dev/null
                echo "${TIMESTAMP} AUTO-FIX: Cleaned old logs on ${mount}" >>
"${LOG_FILE}"
            fi
        fi
    done
```

Why this is production-grade:

- `set -euo pipefail` — fail on errors, undefined variables, pipe failures
- Excludes tmpfs/devtmpfs (virtual filesystems)
- Auto-remediation for log buildup
- Structured logging for audit
- Slack alerting with context

Q8. Explain Linux process management, signals, and systemd.

Answer:

Process lifecycle:

```
fork() → exec() → [running] → exit() → [zombie] → wait() → [reaped]
```

Key signals:

Signal	Number	Default Action	Use Case
`SIGTERM`	15	Graceful termination	`kill <PID>` — ask process to clean up and exit
`SIGKILL`	9	Immediate kill	`kill -9` — last resort, no cleanup
`SIGHUP`	1	Hangup / reload	Reload config without restart (nginx, HAProxy)
`SIGUSR1/2`	10/12	User-defined	Log rotation trigger, debug dump

Signal	Number	Default Action	Use Case
`SIGSTOP`	19	Pause process	Freeze for debugging
`SIGCONT`	18	Resume process	Continue after SIGSTOP

Why `kill -9` should be a last resort: It skips signal handlers — process can't flush buffers, close DB connections, release locks, or write final logs. Always try `SIGTERM` first, wait 30s, then `SIGKILL` if needed.

Systemd service management:

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application Service
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=notify
User=appuser
Group=appgroup
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/start.sh
ExecStop=/opt/myapp/bin/stop.sh
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
StartLimitBurst=3
StartLimitIntervalSec=60
# Security hardening
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/opt/myapp/data /var/log/myapp

[Install]
WantedBy=multi-user.target
```

Key systemd commands:

```
systemctl start/stop/restart/reload myapp
systemctl enable myapp          # Start on boot
systemctl status myapp          # Current state + recent logs
journalctl -u myapp -f          # Live logs
journalctl -u myapp --since "1 hour ago"  # Recent logs
systemctl list-units --failed   # All failed services
```

Q9. Explain Linux networking concepts relevant to DevOps troubleshooting.

Answer:

Key concepts and commands:

```
# DNS resolution
dig example.com              # Full DNS query details
nslookup example.com         # Simple lookup
cat /etc/resolv.conf         # DNS server configuration
nscd -i hosts                # Flush DNS cache

# Network interfaces and IP
ip addr show                 # All interfaces and IPs
ip route show                # Routing table
ip route get 10.0.1.50       # Which interface/route reaches this IP?

# Port and connection debugging
ss -tulnp                    # All listening ports with process names
ss -ant | awk '{print $1}' | sort | uniq -c | sort -rn  # Connection state
distribution
# Too many TIME_WAIT → connection reuse issue
# Too many CLOSE_WAIT → application not closing connections

# Connectivity testing
curl -v https://api.example.com    # HTTP with full headers/TLS handshake
telnet db-host 5432                # Can I reach the DB port?
nc -zv api-host 443                # Port connectivity check
traceroute api.example.com         # Hop-by-hop path
mtr api.example.com                # Combined traceroute + ping (live)

# Firewall
iptables -L -n -v            # List all firewall rules with counters
iptables -L -n -v | grep DROP  # Find blocked traffic
# On newer systems:
nft list ruleset              # nftables rules
```

Common networking issues in DevOps:

Issue	Diagnosis	Fix
DNS resolution failure	`dig` returns NXDOMAIN or timeout	Check `/etc/resolv.conf`, VPC DNS settings
Connection refused	Port not open or service not running	`ss -tulnp` to verify; check security groups

Issue	Diagnosis	Fix
Connection timeout	Firewall/SG blocking or routing issue	`traceroute`, check SG rules, NACLs, route tables
Intermittent timeouts	MTU issues or packet loss	`ping -M do -s 1472` to test MTU; check `mtr`
TLS handshake failure	Certificate expired or mismatch	`openssl s_client -connect host:443`

SECTION 3: TERRAFORM DEEP-DIVE (3/5 Rating Required)

Q10. How do you structure Terraform for a multi-environment production setup?

Answer:

My preferred structure:

```
terraform/
├── modules/                    # Reusable modules
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── ecs-service/
│   ├── rds/
│   └── monitoring/
├── environments/
│   ├── dev/
│   │   ├── main.tf             # Module calls with dev-specific params
│   │   ├── variables.tf
│   │   ├── terraform.tfvars    # Dev values
│   │   └── backend.tf          # Dev state bucket
│   ├── staging/
│   │   ├── main.tf
│   │   ├── terraform.tfvars
│   │   └── backend.tf
│   └── prod/
│       ├── main.tf
```

```
│       ├── terraform.tfvars
│       └── backend.tf
└── global/                    # Shared resources (IAM, Route53)
    ├── iam/
    └── dns/
```

Key decisions and why:

1. **Separate state per environment** — A bad `terraform apply` in dev can't corrupt prod state
2. **Modules for reusability** — VPC module used across all environments with different CIDRs
3. **Workspaces vs directories** — I prefer directories over workspaces because:
   - Each env can have different module versions
   - Different backends and state isolation
   - Clearer Git history per environment
4. **Remote backend with locking:**

```
# backend.tf
terraform {
  backend "s3" {
    bucket         = "company-terraform-state"
    key            = "prod/network/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock"
  }
}
```

Q11. How do you manage Terraform state safely in a team?

Answer:

State management best practices:

```
# 1. Remote state in S3 with DynamoDB locking
# Two engineers can't run terraform apply simultaneously

# 2. State encryption — S3 SSE-KMS
# State contains passwords, IPs, resource IDs
```

```
# 3. Never edit state manually
terraform state mv    # Rename resources
terraform state rm    # Remove from state (not infra)
terraform import      # Import existing resources
# Always in a change-controlled process

# 4. State file per component
# Don't put entire infrastructure in one state file
# Split by: network, compute, database, monitoring
# Reason: blast radius — a mistake in monitoring won't affect networking
```

Handling state drift:

```
# Detect drift
terraform plan        # Shows what's different between state and reality

# Common causes of drift:
# - Manual console changes (biggest offender)
# - Auto-scaling events
# - AWS-managed updates (RDS minor version patches)

# Prevention:
# - SCPs denying console access to managed resources
# - terraform plan in CI on every PR — catches drift early
# - Weekly scheduled terraform plan for drift detection
```

Q12. Explain Terraform modules, data sources, and dependency management.

Answer:

Module design principles:

```
# modules/ecs-service/main.tf
resource "aws_ecs_service" "this" {
  name            = var.service_name
  cluster         = var.cluster_id
  task_definition = aws_ecs_task_definition.this.arn
  desired_count   = var.desired_count
  launch_type     = "FARGATE"

  network_configuration {
    subnets          = var.private_subnet_ids
    security_groups  = [aws_security_group.service.id]
    assign_public_ip = false
```

```
  }
}

# Module creates its own security group — encapsulation
resource "aws_security_group" "service" {
  name_prefix = "${var.service_name}-"
  vpc_id      = var.vpc_id

  ingress {
    from_port       = var.container_port
    to_port         = var.container_port
    protocol        = "tcp"
    security_groups = [var.alb_security_group_id]  # Only from ALB
  }
}
```

Data sources — query existing infrastructure:

```
# Look up existing VPC (managed by another team/state)
data "aws_vpc" "main" {
  tags = { Name = "production-vpc" }
}

# Look up latest AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

# Cross-state reference — read outputs from another state file
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "company-terraform-state"
    key    = "prod/network/terraform.tfstate"
    region = "us-east-1"
  }
}

# Use: data.terraform_remote_state.network.outputs.vpc_id
```

Dependency management:

- **Implicit:** Terraform auto-detects dependencies from references (`var.vpc_id` creates dependency)

- **Explicit:** `depends_on` for hidden dependencies (e.g., IAM policy must exist before Lambda)
- **Module versioning:** Pin module versions in `source` for reproducibility

```
module "vpc" {
  source  = "git::https://gitlab.company.com/modules/vpc.git?ref=v2.3.1"
}
```

End of Part 1 — Continue to Part 2 for AWS, SRE practices, and scenario questions